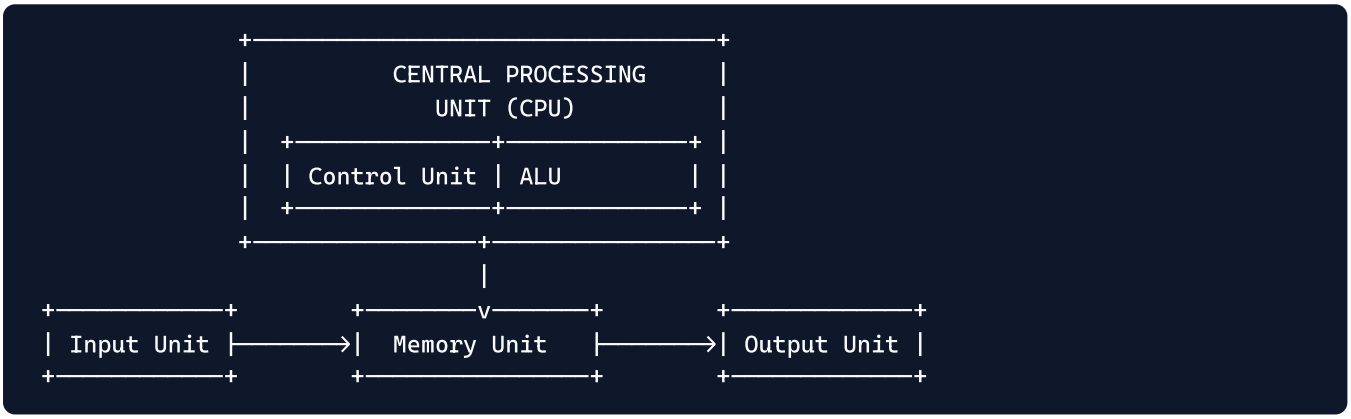
MPCA Module 1 — 3 Mark Questions & Answers

Q1. Explain the functional units of a computer system with a diagram.

A computer system consists of five functional units:

1. **Input Unit:** Accepts data and instructions from the user/environment (e.g., Keyboard, Mouse).
2. **Memory Unit:** Stores programs and data. Divided into Main Memory (RAM) and Secondary Memory (Disk).
3. **Arithmetic and Logic Unit (ALU):** Performs arithmetic calculations (+, -, ×) and logical operations (AND, OR).
4. **Control Unit (CU):** Directs and coordinates all hardware operations by issuing control signals.
5. **Output Unit:** Sends processed results back to the user (e.g., Monitor, Printer).



Q2. Differentiate between Computer Architecture and Computer Organization (any 3 points).

Feature	Computer Architecture	Computer Organization
Focus	*What* the computer does (programmer's view).	*How* the system implements it (hardware view).
Attributes	Instruction set, addressing modes, bit width.	Memory technology, clock speed, bus signals.
Visibility	Visible to the assembly programmer.	Invisible to the high-level programmer.
Example	8086 has a 16-bit instruction set.	8086 uses a 6-byte instruction prefetch queue.

Q3. Explain the instruction cycle (Fetch-Decode-Execute-Store) briefly.

The **Instruction Cycle** is the continuous process by which a CPU processes machine instructions:

1. **Fetch:** The Control Unit reads the instruction byte from main memory using the address in the Program Counter (PC / IP).
2. **Decode:** The Control Unit interprets the instruction opcode to determine the required operation and operands.
3. **Execute:** The ALU performs the specified operation (e.g., addition, bitwise logic).

4. **Store (Write-Back):** The result is written back to a destination register or memory location, and PC/IP is updated.

Q4. List and explain any three performance measures of computer architecture.

1. **CPU Execution Time:** The total time the processor spends actively computing a task. Lower execution time means better performance.
   * `CPU Time = Instruction Count × CPI × Clock Cycle Time`
2. **CPI (Cycles Per Instruction):** The average number of clock cycles needed per instruction. Lower CPI indicates a more efficient architecture (achieved via pipelining).
3. **MIPS (Millions of Instructions Per Second):** Indicates raw processing speed by measuring how many million instructions the CPU executes per second.

Q5. Explain register indirect and indexed addressing modes with examples.

1. **Register Indirect Addressing Mode:**
   * The effective address of the operand is held inside a base or index register (`BX` , `BP` , `SI` , or `DI`).
   * **Syntax:** `MOV AX, [BX]`
   * **Example:** Reads 16-bit data from the memory location pointed to by `DS:BX` .
2. **Indexed Addressing Mode:**
   * The effective address is calculated by adding a displacement constant to an index register (`SI` or `DI`).
   * **Syntax:** `MOV AX, [SI + displacement]`
   * **Example:** `MOV AX, [SI + 2000H]` calculates `Effective Address = SI + 2000H` to access array elements.

Q6. Explain the memory organization (segmented) of the 8086 with the physical address formula.

The 8086 features a **20-bit address bus**, allowing it to address **1 MB (2^{20} bytes)** of memory. Since internal registers are only 16-bit, memory is organized into logical **64 KB segments**:
* **CS (Code Segment):** Stores instruction code.
* **DS (Data Segment):** Stores variables and constants.
* **SS (Stack Segment):** Stores stack frames and local variables.
* **ES (Extra Segment):** Stores string destination data.

Physical Address Calculation Formula:
* `Physical Address = (Segment Register × 10H) + Offset`
* *Example:* If `CS = 2000H` and `IP = 1234H` , `Physical Address = 20000H + 1234H = 21234H` .

Q7. What are the advantages of the Von Neumann model? (any 3)

1. **Simplicity and Low Cost:** Uses a single shared memory and bus system for both code and data, simplifying hardware design and reducing manufacturing costs.
2. **Flexibility:** Memory space is shared dynamically between code and data. Programs can allocate more memory to data or code as needed without physical constraints.
3. **Stored Program Capability:** Programs can be easily loaded, modified, and swapped in main memory without physically rewiring the circuit.

Q8. Explain any three arithmetic instructions with syntax and example.

1. **ADD (Addition):** Adds source to destination and stores result in destination.
   * **Syntax:** `ADD destination, source`
   * **Example:** `ADD AX, BX` (`AX = AX + BX`)
2. **SUB (Subtraction):** Subtracts source from destination.
   * **Syntax:** `SUB destination, source`
   * **Example:** `SUB CX, 0005H` (`CX = CX - 5`)
3. **INC (Increment):** Adds 1 to the specified register or memory operand.
   * **Syntax:** `INC destination`
   * **Example:** `INC SI` (`SI = SI + 1`)

Q9. Explain PUSH and POP instructions with examples.

1. **PUSH Instruction:**
   * Pushes a 16-bit operand onto the top of the stack.
   * Decrements Stack Pointer (`SP = SP - 2`), then stores data at `SS:SP`.
   * **Example:** `PUSH AX` (Saves contents of `AX` onto stack).
2. **POP Instruction:**
   * Pops a 16-bit operand from the top of the stack into a register or memory location.
   * Copies data from `SS:SP` into destination, then increments Stack Pointer (`SP = SP + 2`).
   * **Example:** `POP BX` (Restores top of stack into `BX`).

Q10. Differentiate between JMP and CALL instructions.

Feature	JMP (Jump) Instruction	CALL Instruction
Purpose	Unconditional control transfer to a target address.	Invokes a subroutine (function).
Return Address	Does NOT save return address.	Pushes the return address (`CS` and `IP`) onto the stack.
Return Mechanism	Cannot return to the calling point.	Subroutine uses `RET` to return to the calling location.
Use Case	Loops, goto-style branches.	Modular function calls.

Q11. Explain SHL, SHR and SAR instructions with examples.

1. **SHL (Shift Logical Left):** Shifts bits left by count, filling empty LSBs with 0. MSB goes into Carry Flag (CF). Equivalent to multiplying by 2.
   * **Example:** `SHL AX, 1`
2. **SHR (Shift Logical Right):** Shifts bits right by count, filling empty MSBs with 0. LSB goes into CF. Used for unsigned division by 2.
   * **Example:** `SHR BX, 1`
3. **SAR (Shift Arithmetic Right):** Shifts bits right while **preserving the sign bit (MSB)**. Used for signed division by 2.

* **Example:** `SAR CX, 1`

Q12. Explain the relationship: Clock Rate, Clock Cycle Time, and CPU Time.

- **Clock Cycle Time (T_c):** The duration of one clock tick (e.g., 0.5 ns).
- **Clock Rate (f):** The frequency of the clock oscillator (e.g., 2 GHz). They are reciprocals: $$\text{Clock Rate} = \frac{1}{\text{Clock Cycle Time}}$$
- **CPU Time:** The product of total clock cycles and cycle time: $$\text{CPU Time} = \text{CPU Clock Cycles} \times \text{Clock Cycle Time} = \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}$$
- *Impact:* Increasing Clock Rate reduces Clock Cycle Time, directly reducing CPU Time and improving speed.
